==========

DOCUMENTACIÓN COMPLETA DEL PROYECTO "REPARA MI BARRIO"

Versión 0.1.0

TABLA DE CONTENIDOS

1. Introducción
2. Descripción General del Proyecto
3. Características Principales
4. Arquitectura del Sistema
5. Tecnologías Utilizadas
6. Estructura de Archivos
7. Componentes del Sistema
8. Modelos de Datos
9. Flujo de Navegación
10. Pantallas de la Aplicación
11. Instalación y Configuración
12. Guía de Uso
13. Desarrollo y Personalización
14. Consideraciones Técnicas
15. Mejoras Futuras

==========

1. INTRODUCCIÓN

"Repara mi Barrio" es una aplicación web prototipo diseñada para facilitar la comunicación entre los ciudadanos y las autoridades municipales. Permite a los usuarios reportar problemas de infraestructura y servicios públicos en su comunidad, realizar seguimiento del estado de sus reportes y contribuir activamente a la mejora de su barrio.

Este documento proporciona una guía completa sobre la arquitectura, funcionalidades y uso del sistema.

==========

2. DESCRIPCIÓN GENERAL DEL PROYECTO

Propósito:

Facilitar la participación ciudadana en el mantenimiento y mejora de los espacios públicos mediante un sistema de reportes georreferenciados.

Alcance:

- Registro y autenticación de usuarios
- Creación de reportes con fotografías y ubicación
- Visualización de reportes en un mapa interactivo
- Seguimiento del estado de los reportes
- Gestión de perfil de usuario
- Sistema de notificaciones
- Categorización de problemas por tipo

Beneficios:

✓ Mejora la comunicación ciudadano-gobierno ✓ Permite respuesta rápida a problemas urbanos ✓ Transparencia en la gestión de reportes ✓ Participación ciudadana activa ✓ Geolocalización precisa de problemas

==========

3. CARACTERÍSTICAS PRINCIPALES

3.1 Sistema de Reportes

• Creación de reportes con título, descripción y fotografía • Categorización por tipo de problema • Geolocalización automática o manual • Estado del reporte (Pendiente, En Proceso, Resuelto) • Línea de tiempo de seguimiento

3.2 Mapa Interactivo

• Visualización de todos los reportes en el mapa • Marcadores diferenciados por categoría y estado • Leyenda visual de categorías y estados • Controles de zoom y ubicación actual • Selección de reportes en el mapa

3.3 Gestión de Usuario

• Registro con email y contraseña • Inicio de sesión seguro • Perfil personalizable con avatar • Estadísticas de reportes del usuario • Historial de reportes propios

3.4 Categorías de Reportes

• Baches (Infraestructura vial) • Basura (Gestión de residuos) • Alumbrado (Servicios de iluminación) • Agua (Fugas y servicios hídricos) • Seguridad (Problemas de seguridad ciudadana)

3.5 Sistema de Notificaciones

• Alertas de cambio de estado en reportes • Notificaciones de resolución • Actualizaciones del sistema

==========

4. ARQUITECTURA DEL SISTEMA

Patrón de Diseño:

Arquitectura de aplicación de página única (SPA - Single Page Application) con gestión de estado mediante React Hooks.

Capas del Sistema:

┌─────────────────────────────────┐ │ CAPA DE PRESENTACIÓN │ │ (Componentes React + UI) │ ├─────────────────────────────────┤ │ CAPA DE LÓGICA │ │ (Estado, Navegación, Hooks) │ ├─────────────────────────────────┤ │ CAPA DE DATOS │ │ (Modelos, Mock Data) │ └─────────────────────────────────┘

Flujo de Datos:

Usuario → Componentes → Estado Local → Actualización de UI

==========

5. TECNOLOGÍAS UTILIZADAS

Frontend:

• React 18.3.1 - Biblioteca principal de UI • TypeScript - Tipado estático • Vite 6.3.5 - Herramienta de construcción y desarrollo • React DOM 18.3.1 - Renderizado de React

Librerías de UI:

• Radix UI - Componentes accesibles primitivos

- Accordion, Alert Dialog, Avatar, Checkbox, Dialog
- Dropdown Menu, Hover Card, Label, Navigation Menu
- Popover, Radio Group, Select, Slider, Switch
- Tabs, Toggle, Tooltip, y más

• Lucide React 0.487.0 - Iconos SVG • Tailwind CSS - Framework de estilos utility-first • Class Variance Authority - Gestión de variantes de clase • clsx - Utilidad para clases condicionales • Tailwind Merge - Merge de clases Tailwind

Componentes Especializados:

• Sonner 2.0.3 - Sistema de notificaciones toast • React Hook Form 7.55.0 - Gestión de formularios • React Day Picker 8.10.1 - Selector de fechas • Recharts 2.15.2 - Gráficos y visualizaciones • Embla Carousel 8.6.0 - Carruseles responsive • CMDK 1.1.1 - Command palette • Input OTP 1.4.2 - Campos de entrada OTP • Vaul 1.1.2 - Drawers móviles • React Resizable Panels 2.1.7 - Paneles redimensionables • Next Themes 0.4.6 - Sistema de temas

Herramientas de Desarrollo:

• @vitejs/plugin-react-swc 3.10.2 - Compilador rápido de React • @types/node 20.10.0 - Tipos de TypeScript para Node.js

==========

6. ESTRUCTURA DE ARCHIVOS

Repara mi Barrio Prototipo/ │ ├── index.html # Punto de entrada HTML ├── package.json # Dependencias y scripts ├── vite.config.ts # Configuración de Vite ├── README.md # Documentación básica ├── DOCUMENTACION.txt # Esta documentación │ └── src/ # Código fuente ├── main.tsx # Punto de entrada de React ├── App.tsx # Componente principal ├── index.css # Estilos globales │ ├── components/ # Componentes React │ ├── ui/ # Componentes UI reutilizables │ │ ├── accordion.tsx │ │ ├── alert-dialog.tsx │ │ ├── avatar.tsx │ │ ├── badge.tsx │ │ ├── button.tsx │ │ ├── card.tsx │ │ ├── checkbox.tsx │ │ ├── dialog.tsx │ │ ├── input.tsx │ │ ├── label.tsx │ │ ├── select.tsx │ │ ├── textarea.tsx │ │ ├── tooltip.tsx │ │ └── ... (más componentes UI) │ │ │ ├── figma/ # Componentes de integración │ │ └── ImageWithFallback.tsx │ │ ├── WelcomeScreen.tsx # Pantalla de bienvenida │ ├── LoginScreen.tsx # Pantalla de inicio de sesión │ ├── RegisterScreen.tsx # Pantalla de registro │ ├── MapScreen.tsx # Pantalla del mapa │ ├── CreateReportScreen.tsx # Crear nuevo reporte │ ├── ReportsListScreen.tsx # Lista de reportes │ ├── ReportDetailScreen.tsx # Detalle de reporte │ ├── ProfileScreen.tsx # Perfil de usuario │ ├── MyReportsScreen.tsx # Mis reportes │ ├── EditProfileScreen.tsx # Editar perfil │ ├── NotificationsScreen.tsx #

Notificaciones │ ├── SettingsScreen.tsx # Configuración │ ├── SupportScreen.tsx # Soporte │ ├── AboutScreen.tsx # Acerca de │ └── BottomNav.tsx # Navegación inferior │ ├── lib/ # Lógica y utilidades │ └── data.ts # Modelos y datos mock │ ├── styles/ # Estilos adicionales │ └── globals.css │ ├── guidelines/ # Guías de diseño │ └── Guidelines.md │ └── Attributions.md # Atribuciones

==========

7. COMPONENTES DEL SISTEMA

7.1 Componente Principal (App.tsx)

Responsabilidades: • Gestión del estado global de navegación • Control del flujo entre pantallas • Manejo del historial de navegación • Gestión del tab activo en navegación inferior

Estados Principales:

- currentScreen: Pantalla actual mostrada
- activeTab: Tab activo en la navegación principal
- selectedReport: Reporte seleccionado para ver detalle
- navigationHistory: Historial de navegación para el botón atrás

7.2 Pantallas de Autenticación

WelcomeScreen:

- Pantalla inicial con logo y opciones
- Navegación a Login o Registro
- Diseño atractivo con gradiente e imagen

LoginScreen:

- Formulario de inicio de sesión
- Campos: email y contraseña
- Opción de recuperar contraseña
- Enlace a registro

RegisterScreen:

- Formulario de registro de nuevo usuario
- Campos: nombre, email, contraseña, confirmación
- Validación de datos
- Términos y condiciones

7.3 Pantallas Principales

MapScreen:

- Mapa interactivo simulado
- Marcadores de reportes con iconos por categoría
- Indicadores de estado con puntos de color
- Leyenda de categorías y estados
- Controles de zoom y ubicación
- Card de detalle al seleccionar reporte
- Botón flotante para crear nuevo reporte

CreateReportScreen:

- Formulario completo de reporte
- Upload de foto
- Campos de título y descripción
- Selector de categoría con botones visuales
- Selector de ubicación con mini-mapa
- Botón de ubicación actual
- Validación de campos

ReportsListScreen:

- Lista de todos los reportes
- Filtros por categoría y estado
- Barra de búsqueda
- Cards de reporte con imagen, título, categoría y estado
- Navegación a detalle de reporte

ReportDetailScreen:

- Vista completa de un reporte
- Imagen grande
- Información detallada
- Ubicación en mapa
- Timeline de estado
- Botones de acción

7.4 Pantallas de Usuario

ProfileScreen:

- Información del usuario
- Avatar y nombre
- Estadísticas de reportes
- Opciones: Mis reportes, Editar perfil, Notificaciones
- Configuración y soporte
- Cerrar sesión

MyReportsScreen:

- Lista de reportes del usuario
- Filtros de estado
- Estadísticas personales
- Acceso rápido a detalles

EditProfileScreen:

- Formulario de edición de perfil
- Cambio de avatar
- Actualización de datos personales
- Botón de guardar cambios

NotificationsScreen:

- Lista de notificaciones
- Marcado de leído/no leído
- Filtrado por tipo
- Eliminación de notificaciones

SettingsScreen:

- Configuración de cuenta
- Preferencias de notificaciones
- Configuración de privacidad
- Idioma y región

SupportScreen:

- Preguntas frecuentes
- Formulario de contacto
- Información de soporte

AboutScreen:

- Información de la aplicación
- Versión actual
- Términos y condiciones
- Política de privacidad

7.5 Componentes de UI Reutilizables

BottomNav:

- Navegación inferior con 3 tabs
- Iconos y labels
- Indicador de tab activo
- Responsive design

Button:

- Variantes: default, outline, ghost, destructive
- Tamaños: default, sm, lg, icon
- Estados: hover, active, disabled

Card:

- Contenedor con sombra
- Header, Content, Footer
- Diseño flexible

Input:

- Campo de texto estándar
- Variantes de validación
- Placeholder y label

Badge:

- Etiquetas de estado y categoría
- Colores personalizables
- Tamaños variados

Dialog:

- Modales para confirmaciones
- Overlay con animación
- Contenido personalizable

==========

8. MODELOS DE DATOS

8.1 Report (Reporte)

interface Report { id: string; // Identificador único title: string; // Título del reporte description: string; // Descripción detallada category: CategoryType; // Categoría del problema status: StatusType; // Estado actual location: { // Información de ubicación lat: number; // Latitud lng: number; // Longitud address: string; // Dirección legible }; date: string; // Fecha de creación imageUrl?: string; // URL de la imagen (opcional) user: { // Información del usuario name: string; // Nombre del usuario avatar?: string; // Avatar (opcional) }; timeline?: Array<{ // Línea de tiempo (opcional) status: string; // Estado en ese momento date: string; // Fecha del cambio description: string; // Descripción del cambio }>; }

8.2 Tipos de Categoría

type CategoryType = | 'baches' // Problemas de pavimento | 'basura' // Gestión de residuos | 'alumbrado' // Iluminación pública | 'agua' // Servicios hídricos | 'seguridad' // Seguridad ciudadana

Configuración de Categorías: { id: 'baches', name: 'Baches', color: '#F97316', // Naranja icon: '🕳️' }

8.3 Tipos de Estado

type StatusType = | 'pendiente' // Recién reportado | 'en-proceso' // En atención | 'resuelto' // Completado

Configuración de Estados: { 'pendiente': { name: 'Pendiente', color: '#EAB308' // Amarillo }, 'en-proceso': { name: 'En proceso', color: '#3B82F6' // Azul }, 'resuelto': { name: 'Resuelto', color: '#22C55E' // Verde } }

8.4 Estadísticas de Usuario

interface UserStats { totalReports: number; // Total de reportes creados activeReports: number; // Reportes activos (pendiente + en proceso) resolvedReports: number; // Reportes resueltos }

==========

9. FLUJO DE NAVEGACIÓN

9.1 Flujo de Autenticación

[Bienvenida] ↓ ├→ [Login] → [Pantalla Principal] │ ↑ │ └ [Recuperar Contraseña] │ └→ [Registro] → [Pantalla Principal]

9.2 Flujo Principal de la App

[Pantalla Principal] │ ├ Tab Mapa │ ├→ [Ver Reporte en Mapa] │ │ ↓ │ │ [Detalle de Reporte] │ │ │ └→ [Crear Nuevo Reporte] │ ↓ │ [Formulario] │ ↓ │ [Confirmación] │ ↓ │ [Volver al Mapa] │ ├ Tab Reportes │ ├→ [Lista de Reportes] │ │ ↓ │ │ [Detalle de Reporte] │ │ │ └→ [Filtros y Búsqueda] │ └ Tab Perfil ├→ [Ver Perfil] ├→ [Mis Reportes] ├→ [Editar Perfil] ├→ [Notificaciones] ├→ [Configuración] ├→ [Soporte] ├→ [Acerca de] └→ [Cerrar Sesión] → [Bienvenida]

9.3 Navegación con Botón Atrás

El sistema mantiene un historial de navegación que permite:

- Volver a la pantalla anterior
- Preservar el estado del tab activo
- Evitar salir accidentalmente de la app

==========

10. PANTALLAS DE LA APLICACIÓN

10.1 Pantalla de Bienvenida

Elementos:

- Logo de la aplicación (icono de casa)
- Título: "Repara mi Barrio"
- Descripción breve
- Imagen ilustrativa de ciudad
- Botón "Iniciar sesión" (verde principal)
- Botón "Registrarse" (outline verde)

Colores:

- Fondo: Gradiente verde claro a amarillo (#f0fdf4 a #fef3c7)

- Botones: Verde #27AE60

10.2 Pantalla de Login

Elementos:

- Header con botón atrás
- Título: "Iniciar Sesión"
- Campo de email
- Campo de contraseña
- Checkbox "Recordarme"
- Link "¿Olvidaste tu contraseña?"
- Botón "Iniciar Sesión"
- Link a registro: "¿No tienes cuenta? Regístrate"

Validaciones:

- Email válido
- Contraseña no vacía

10.3 Pantalla de Registro

Elementos:

- Header con botón atrás
- Título: "Crear Cuenta"
- Campo de nombre completo
- Campo de email
- Campo de contraseña
- Campo de confirmar contraseña
- Checkbox de términos y condiciones
- Botón "Registrarse"
- Link a login: "¿Ya tienes cuenta? Inicia sesión"

Validaciones:

- Nombre no vacío
- Email válido y único
- Contraseñas coinciden
- Aceptación de términos

10.4 Pantalla del Mapa

Elementos:

- Mapa simulado con calles
- Marcadores de reportes:
  - Círculo de color por categoría
  - Punto de estado en esquina
  - Icono representativo
- Leyenda flotante (plegable):
  - Categorías con colores
  - Estados con indicadores
- Controles de mapa:
  - Zoom in
  - Zoom out
  - Centrar en ubicación actual
- Card de reporte seleccionado:
  - Imagen miniatura
  - Título y descripción
  - Badges de categoría y estado
  - Botón "Ver detalle"
  - Botón "Cerrar"
- Botón flotante "+" para crear reporte
- Barra de navegación inferior

Interacciones:

- Click en marcador muestra card
- Click en "Ver detalle" abre pantalla de detalle
- Click en "+" abre formulario de creación
- Controles de zoom funcionales (simulados)

10.5 Pantalla de Crear Reporte

Elementos:

- Header con botón atrás y título
- Sección de foto:
  - Área de drop/upload

    - Icono de cámara
    - Texto "Toca para agregar foto"
- Campo de título
- Campo de descripción (textarea)
- Selector de categoría:
    - Botones con iconos y colores
    - Selección única
    - Efecto visual al seleccionar
- Sección de ubicación:
    - Mini-mapa placeholder
    - Dirección seleccionada
    - Botón "Usar mi ubicación"
- Botón "Enviar Reporte" (verde)

Validaciones:

- Título obligatorio
- Descripción obligatoria
- Categoría obligatoria
- Ubicación recomendada

Flujo:

1. Usuario completa formulario
2. Sistema valida datos
3. Muestra notificación de éxito
4. Redirige a mapa principal

10.6 Pantalla de Lista de Reportes

Elementos:

- Header con título "Reportes"
- Barra de búsqueda
- Filtros:
    - Por categoría (chips)
    - Por estado (dropdown)
- Lista de cards de reportes:
    - Imagen
    - Título
    - Descripción breve
    - Badges de categoría y estado
    - Fecha
    - Ubicación
- Barra de navegación inferior

Funcionalidad:

- Búsqueda por texto en tiempo real
- Filtrado múltiple
- Scroll infinito (simulado)
- Click en card abre detalle

10.7 Pantalla de Detalle de Reporte

Elementos:

- Header con botón atrás
- Imagen principal (full-width)
- Título del reporte
- Badges de categoría y estado
- Información del usuario:
    - Avatar
    - Nombre
    - Fecha de reporte
- Descripción completa
- Sección de ubicación:
    - Mapa pequeño
    - Dirección completa
    - Botón "Ver en mapa"
- Timeline de estado:
    - Lista cronológica
    - Iconos de estado
    - Fechas y descripciones
- Botones de acción:
    - "Reportar problema similar"
    - "Compartir"

10.8 Pantalla de Perfil

Elementos:

- Header con título "Perfil"
- Sección de usuario:
  - Avatar grande (editable)
  - Nombre
  - Email
  - Botón "Editar perfil"
- Estadísticas:
  - Total de reportes
  - Reportes activos
  - Reportes resueltos
- Opciones del menú:
  - Mis reportes
  - Notificaciones (con badge de contador)
  - Configuración
  - Soporte
  - Acerca de
  - Cerrar sesión (rojo)
- Barra de navegación inferior

10.9 Pantalla de Mis Reportes

Elementos:

- Header con botón atrás y título
- Tabs de filtro:
  - Todos
  - Pendientes
  - En proceso
  - Resueltos
- Estadísticas rápidas
- Lista de reportes propios
- Estado vacío si no hay reportes

10.10 Pantalla de Editar Perfil

Elementos:

- Header con botón atrás
- Botón de cambiar avatar
- Campo de nombre
- Campo de email
- Campo de teléfono (opcional)
- Campo de ubicación predeterminada
- Botón "Guardar cambios"
- Botón "Cancelar"

10.11 Pantalla de Notificaciones

Elementos:

- Header con botón atrás
- Contador de no leídas
- Botón "Marcar todas como leídas"
- Lista de notificaciones:
  - Icono según tipo
  - Título
  - Descripción
  - Fecha
  - Indicador de leído/no leído
- Estado vacío si no hay notificaciones

Tipos de notificaciones:

- Cambio de estado de reporte
- Comentarios en reportes
- Respuestas oficiales
- Actualizaciones del sistema

10.12 Pantalla de Configuración

Elementos:

- Header con botón atrás
- Secciones:

- Cuenta
  - Cambiar contraseña
  - Email de contacto
- Notificaciones
  - Push notifications
  - Email notifications
  - Frecuencia de actualizaciones
- Privacidad
  - Perfil público/privado
  - Mostrar ubicación
- Preferencias
  - Idioma
  - Tema (claro/oscuro)
- Datos
  - Descargar mis datos
  - Eliminar cuenta

10.13 Pantalla de Soporte

Elementos:

- Header con botón atrás
- Sección de FAQ:
  - Preguntas frecuentes expandibles
- Formulario de contacto:
  - Asunto
  - Mensaje
  - Botón enviar
- Información de contacto:
  - Email de soporte
  - Horarios de atención
  - Redes sociales

10.14 Pantalla de Acerca de

Elementos:

- Header con botón atrás
- Logo de la aplicación
- Versión actual
- Descripción del proyecto
- Equipo de desarrollo
- Enlaces a:
  - Términos y condiciones
  - Política de privacidad
  - Licencias de código abierto
- Botón "Visitar sitio web"

==========:

11. INSTALACIÓN Y CONFIGURACIÓN

11.1 Requisitos Previos

- Node.js versión 20.x o superior
- npm (incluido con Node.js) o yarn
- Navegador web moderno (Chrome, Firefox, Safari, Edge)
- Editor de código (recomendado: VS Code)

11.2 Instalación Paso a Paso

Paso 1: Clonar o Descargar el Proyecto

Si tienes el proyecto en Git: $ git clone [URL_DEL_REPOSITORIO] $ cd "Repara mi Barrio Prototipo"

Si tienes el archivo comprimido:

- Extraer el contenido
- Abrir terminal en la carpeta del proyecto

Paso 2: Instalar Dependencias

$ npm install

Este comando instalará todas las dependencias listadas en package.json.

Tiempo estimado: 2-5 minutos (depende de la conexión a internet)

Paso 3: Iniciar el Servidor de Desarrollo

$ npm run dev

La aplicación estará disponible en: http://localhost:5173

Vite mostrará un mensaje similar a: VITE v6.3.5 ready in 234 ms

➜ Local: http://localhost:5173/ ➜ Network: use --host to expose

Paso 4: Abrir en el Navegador

- Abrir navegador
- Ir a: http://localhost:5173
- La aplicación se recargará automáticamente al hacer cambios

11.3 Construcción para Producción

Para generar una versión optimizada para producción:

$ npm run build

Esto creará una carpeta 'dist' con los archivos optimizados.

Despliegue:

- Subir el contenido de 'dist' a un servidor web
- Configurar el servidor para SPA (todas las rutas a index.html)

11.4 Solución de Problemas

Error: "Cannot find module..." Solución: Ejecutar npm install nuevamente

Error: "Port 5173 already in use" Solución:

- Cerrar otras instancias de Vite
- O cambiar el puerto en vite.config.ts

Error de compilación de TypeScript: Solución:

- Verificar sintaxis en el archivo mencionado
- Revisar tipos de datos

==========:

12. GUÍA DE USO

12.1 Para Usuarios Finales

Primer Uso:

1. Abrir la aplicación
2. En pantalla de bienvenida, seleccionar "Registrarse"
3. Completar formulario:
    - Nombre completo
    - Email válido
    - Contraseña segura (mínimo 6 caracteres)
    - Confirmar contraseña
4. Aceptar términos y condiciones
5. Click en "Registrarse"
6. Se redirige automáticamente a la pantalla principal

Iniciar Sesión:

1. En pantalla de bienvenida, seleccionar "Iniciar sesión"
2. Ingresar email y contraseña
3. (Opcional) Marcar "Recordarme"
4. Click en "Iniciar Sesión"

Crear un Reporte:

1. En cualquier pantalla principal, click en botón "+" (verde)
2. Tomar foto o seleccionar de galería
3. Escribir título descriptivo Ejemplo: "Bache grande en Calle Principal"
4. Escribir descripción detallada Ejemplo: "Bache de aproximadamente 1 metro de diámetro..."
5. Seleccionar categoría (baches, basura, etc.)
6. Confirmar ubicación:
    - Click en "Usar mi ubicación" (recomendado)
    - O seleccionar manualmente en el mapa

7. Click en "Enviar Reporte"
8. Esperar confirmación
9. Ver reporte creado en el mapa

Ver Reportes en el Mapa:

1. Ir al tab "Mapa" (icono de mapa en navegación inferior)
2. Explorar el mapa
3. Cada marcador representa un reporte:
   - Color = Categoría
   - Punto pequeño = Estado (amarillo=pendiente, azul=proceso, verde=resuelto)
4. Click en un marcador
5. Aparece card con información resumida
6. Click en "Ver detalle" para información completa

Ver Detalles de un Reporte:

1. Seleccionar reporte en mapa o lista
2. Ver información completa:
   - Foto
   - Descripción
   - Ubicación exacta
   - Estado actual
   - Timeline de cambios
3. Acciones disponibles:
   - Reportar problema similar
   - Compartir con otros

Ver Lista de Reportes:

1. Ir al tab "Reportes"
2. Ver todos los reportes en formato de lista
3. Usar barra de búsqueda para buscar por texto
4. Filtrar por:
   - Categoría (seleccionar chips)
   - Estado (dropdown)
5. Scroll para ver más reportes
6. Click en card para ver detalle

Ver Mis Reportes:

1. Ir al tab "Perfil"
2. Click en "Mis reportes"
3. Ver estadísticas personales
4. Filtrar por estado:
   - Todos
   - Pendientes
   - En proceso
   - Resueltos
5. Click en reporte para ver detalle

Editar Perfil:

1. Ir al tab "Perfil"
2. Click en "Editar perfil"
3. Modificar información:
   - Cambiar foto de perfil
   - Actualizar nombre
   - Cambiar email
   - Agregar teléfono
4. Click en "Guardar cambios"

Ver Notificaciones:

1. Ir al tab "Perfil"
2. Click en "Notificaciones"
3. Ver lista de actualizaciones
4. Click en notificación para ver detalles
5. Marcar como leída
6. O "Marcar todas como leídas"

Configurar Preferencias:

1. Ir al tab "Perfil"
2. Click en "Configuración"
3. Ajustar:
   - Notificaciones push

- Email de notificaciones
- Privacidad del perfil
- Idioma
- Tema de color

4. Cambios se guardan automáticamente

Cerrar Sesión:

1. Ir al tab "Perfil"
2. Scroll hasta abajo
3. Click en "Cerrar sesión"
4. Confirmar acción
5. Volver a pantalla de bienvenida

12.2 Para Administradores

Monitoreo de Reportes:

- Revisar reportes regularmente
- Actualizar estados según progreso
- Responder a usuarios
- Asignar recursos según prioridad

Gestión de Usuarios:

- Moderar contenido inapropiado
- Gestionar permisos
- Resolver disputas

Análisis de Datos:

- Identificar áreas problemáticas
- Patrones de reportes
- Tiempos de resolución
- Satisfacción de usuarios

==========:

13. DESARROLLO Y PERSONALIZACIÓN

13.1 Estructura de Componentes

Crear un Nuevo Componente:

1. Crear archivo en /src/components/
2. Estructura básica:

import { ArrowLeft } from "lucide-react"; import { Button } from "./ui/button";

interface MiComponenteProps { onNavigate: (screen: string) => void; }

export function MiComponente({ onNavigate }: MiComponenteProps) { return (

<button onClick={() => onNavigate('back')}>

Título

```
  <div className="px-4 py-6">
    {/* Contenido */}
  </div>
</div>
```

); }

3. Importar en App.tsx
4. Agregar al sistema de navegación

13.2 Personalizar Colores

El color principal verde (#27AE60) se puede cambiar:

1. Buscar todas las referencias a #27AE60
2. Reemplazar con el nuevo color
3. Ajustar hover states (#229954)

Para un cambio global, considerar:

- Crear variables CSS en index.css
- Usar sistema de temas de Tailwind

Ejemplo: :root { --primary: #27AE60; --primary-hover: #229954; }

13.3 Agregar Nueva Categoría

1. Abrir src/lib/data.ts
2. Agregar tipo en CategoryType: type CategoryType = ... | 'nueva-categoria';
3. Agregar configuración en array categories: { id: 'nueva-categoria', name: 'Nueva Categoría', color: '#HEXCOLOR', icon: '📍' }
4. Los cambios se reflejan automáticamente en:
   - Selector de categorías
   - Mapa
   - Filtros
   - Leyenda

13.4 Modificar Estados

1. Abrir src/lib/data.ts
2. Agregar nuevo estado en StatusType
3. Actualizar statusConfig con configuración visual
4. Actualizar lógica de timeline si es necesario

13.5 Agregar Validaciones

Ejemplo en CreateReportScreen.tsx:

const handleSubmit = () => { // Validación de título if (!title || title.length < 10) { toast.error("El título debe tener al menos 10 caracteres"); return; }

// Validación de descripción if (!description || description.length < 20) { toast.error("La descripción debe ser más detallada"); return; }

// Validación de categoría if (!selectedCategory) { toast.error("Selecciona una categoría"); return; }

// Validación de foto (opcional) if (!imageUrl) { toast.warning("Es recomendable agregar una foto"); }

// Continuar con el envío... };

13.6 Integrar con Backend

Para conectar con un servidor real:

1. Instalar axios o fetch API
2. Crear servicio de API:

// src/services/api.ts const API_URL = 'https://tu-api.com';

export const api = { // Autenticación login: async (email: string, password: string) => { const response = await fetch(`${API_URL}/auth/login` , { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ email, password }) }); return response.json(); },

// Reportes getReports: async () => { const response = await fetch(`${API_URL}/reports`); return response.json(); },

createReport: async (reportData: any) => { const response = await fetch(`${API_URL}/reports` , { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(reportData) }); return response.json(); },

// ... más endpoints };

3. Actualizar componentes para usar el servicio
4. Agregar gestión de estados de carga y error
5. Implementar manejo de autenticación con tokens

13.7 Agregar Mapa Real

Para integrar un mapa real (ej: Google Maps, Mapbox):

1. Instalar librería de mapas: $ npm install @react-google-maps/api
2. Obtener API key
3. Modificar MapScreen.tsx:

import { GoogleMap, LoadScript, Marker } from '@react-google-maps/api';

export function MapScreen({ onNavigate }: MapScreenProps) { const [map, setMap] = useState(null);

const center = { lat: 40.4168, lng: -3.7038 };

return (<GoogleMap mapContainerStyle={{ width: '100%', height: '100vh' }} center={center} zoom={14} onLoad={setMap} > {mockReports.map(report => (<Marker key={report.id} position={{ lat: report.location.lat, lng: report.location.lng }} onClick={() => setSelectedReport(report)} icon={{ url: getMarkerIcon(report.category), scaledSize: { width: 40, height: 40 } }} />))}); }

13.8 Optimización de Imágenes

Para mejorar el rendimiento con imágenes:

1. Implementar lazy loading:

import { lazy, Suspense } from 'react';

const ImageComponent = lazy(() => import('./ImageComponent'));

<Suspense fallback={}>

2. Comprimir imágenes antes de subir
3. Usar formatos modernos (WebP, AVIF)
4. Implementar thumbnails para listados

13.9 Añadir Tests

Ejemplo de test con Vitest:

// src/components/Button.test.tsx import { describe, it, expect } from 'vitest'; import { render, screen } from '@testing-library/react'; import { Button } from './ui/button';

describe('Button Component', () => { it('renders button with text', () => { render(Click me); expect(screen.getByText('Click me')).toBeInTheDocument(); });

it('calls onClick handler', () => { const handleClick = vi.fn(); render(Click); screen.getByText('Click').click(); expect(handleClick).toHaveBeenCalledOnce(); }); });

13.10 Internacionalización (i18n)

Para soporte multi-idioma:

1. Instalar react-i18next: $ npm install react-i18next i18next

2. Configurar:

// src/i18n/config.ts import i18n from 'i18next'; import { initReactI18next } from 'react-i18next';

i18n .use(initReactI18next) .init({ resources: { es: { translation: { "welcome.title": "Repara mi Barrio", "welcome.description": "Reporta problemas...", // ... más traducciones } }, en: { translation: { "welcome.title": "Fix My Neighborhood", "welcome.description": "Report issues...", // ... más traducciones } } }, lng: "es", fallbackLng: "es", interpolation: { escapeValue: false } });

3. Usar en componentes:

import { useTranslation } from 'react-i18next';

export function WelcomeScreen() { const { t } = useTranslation();

return (

{t('welcome.title')}

{t('welcome.description')}

); }

==========

14. CONSIDERACIONES TÉCNICAS

14.1 Rendimiento

Optimizaciones Actuales: • React 18 con Concurrent Features • Vite para bundling rápido • SWC para compilación veloz • Code splitting automático

Recomendaciones: • Implementar virtualización para listas largas • Usar React.memo para componentes pesados • Lazy loading de imágenes • Service Workers para caché • Optimización de assets con CDN

14.2 Seguridad

Consideraciones: • Validación de entrada en cliente Y servidor • Sanitización de datos de usuario • Protección contra XSS • HTTPS obligatorio en producción • Autenticación con JWT • Rate limiting en API • Validación de tamaño y tipo de archivos • Protección de datos personales (GDPR)

14.3 Accesibilidad (a11y)

Características: • Componentes Radix UI son accesibles por defecto • Soporte de teclado completo • ARIA labels apropiados • Contraste de colores WCAG AA • Focus visible en elementos interactivos

Mejoras Recomendadas: • Screen reader testing • Alto contraste mode • Reducción de animaciones • Navegación por teclado avanzada • Texto alternativo en imágenes

14.4 Responsive Design

Breakpoints: • Mobile: < 768px • Tablet: 768px - 1024px • Desktop: > 1024px

Estrategia: • Mobile-first approach • Tailwind responsive utilities • Navegación adaptativa • Imágenes responsivas • Touch-friendly en móviles

14.5 PWA (Progressive Web App)

Para convertir en PWA:

1. Agregar manifest.json: { "name": "Repara mi Barrio", "short_name": "RMB", "description": "Reporta problemas en tu barrio", "start_url": "/", "display": "standalone", "background_color": "#ffffff", "theme_color": "#27AE60", "icons": [{ "src": "/icon-192.png", "sizes": "192x192", "type": "image/png" }, { "src": "/icon-512.png", "sizes": "512x512", "type": "image/png" }] }
2. Agregar Service Worker para caché offline
3. Registrar en index.html
4. Habilitar "Add to Home Screen"

14.6 SEO

Mejoras recomendadas: • Server-side rendering (Next.js) • Meta tags apropiados • Structured data (JSON-LD) • Sitemap.xml • Robots.txt • Open Graph tags • Twitter Card tags

14.7 Analytics

Integrar herramientas: • Google Analytics para métricas • Hotjar para mapas de calor • Sentry para error tracking • Custom events para acciones clave

14.8 Base de Datos Recomendada

Para producción:

Backend: • Node.js + Express • PostgreSQL o MongoDB • Redis para caché • AWS S3 para imágenes

Esquema sugerido (PostgreSQL):

-- Usuarios CREATE TABLE users (id UUID PRIMARY KEY, email VARCHAR(255) UNIQUE NOT NULL, password_hash VARCHAR(255) NOT NULL, name VARCHAR(255) NOT NULL, avatar_url TEXT, created_at TIMESTAMP DEFAULT NOW());

-- Reportes CREATE TABLE reports (id UUID PRIMARY KEY, user_id UUID REFERENCES users(id), title VARCHAR(255) NOT NULL, description TEXT NOT NULL, category VARCHAR(50) NOT NULL, status VARCHAR(50) DEFAULT 'pendiente', lat DECIMAL(10, 8) NOT NULL, lng DECIMAL(11, 8) NOT NULL, address TEXT NOT NULL, image_url TEXT, created_at TIMESTAMP DEFAULT NOW(), updated_at TIMESTAMP DEFAULT NOW());

-- Timeline CREATE TABLE report_timeline (id UUID PRIMARY KEY, report_id UUID REFERENCES reports(id), status VARCHAR(50) NOT NULL, description TEXT NOT NULL, created_at TIMESTAMP DEFAULT NOW());

-- Notificaciones CREATE TABLE notifications (id UUID PRIMARY KEY, user_id UUID REFERENCES users(id), type VARCHAR(50) NOT NULL, title VARCHAR(255) NOT NULL, description TEXT, read BOOLEAN DEFAULT FALSE, created_at TIMESTAMP DEFAULT NOW());

==========

15. MEJORAS FUTURAS

15.1 Funcionalidades Propuestas

Corto Plazo: • Autenticación real con JWT • Integración con mapa real (Google Maps/Mapbox) • Upload real de imágenes • Notificaciones push • Comentarios en reportes • Sistema de votación (upvotes) • Compartir en redes sociales • Exportar datos

Mediano Plazo: • Dashboard de administración • Analytics avanzados • Reportes duplicados detection • Geofencing para alertas • Chat en vivo con autoridades • Gamificación (badges, puntos) • Modo offline completo • Multi-idioma

Largo Plazo: • Inteligencia artificial para:

- Clasificación automática de reportes
- Detección de duplicados
- Predicción de tiempos de resolución
- Análisis de sentimiento • Integración con sistemas municipales • API pública • App móvil nativa (React Native) • Blockchain para transparencia • Realidad aumentada para reportes

15.2 Optimizaciones Técnicas

• Migrar a Next.js para SSR • Implementar GraphQL • Microservicios architecture • Kubernetes para escalabilidad • CI/CD pipeline • Automated testing (unit, integration, e2e) • Performance monitoring • A/B testing framework • Feature flags

15.3 Diseño UX/UI

• Modo oscuro completo • Personalización de temas • Animaciones mejoradas • Micro-interacciones • Onboarding interactivo • Tour guiado • Mejoras de accesibilidad • Diseño inclusivo

15.4 Integración con Servicios Externos

• Sistemas municipales existentes • Servicios de mapas • Servicios de notificaciones (Firebase, OneSignal) • Almacenamiento en la nube (AWS, Google Cloud) • Servicios de email (SendGrid, Mailgun) • Servicios de SMS • Redes sociales para compartir • Servicios de pago (si aplicable)

15.5 Aspectos Legales y Cumplimiento

• Cumplimiento GDPR • Términos de servicio actualizados • Política de privacidad robusta • Consentimiento de cookies • Moderación de contenido • Reporte de abuso • Verificación de identidad • Protección de menores

15.6 Monetización (Opcional)

• Modelo freemium • Reportes prioritarios (premium) • API de pago para desarrolladores • Dashboard empresarial • Publicidad no intrusiva • Donaciones • Patrocinios municipales

==========

CONCLUSIÓN

"Repara mi Barrio" es una plataforma innovadora que facilita la participación ciudadana en el mantenimiento urbano. Este prototipo demuestra las funcionalidades core del sistema y sirve como base para el desarrollo de una aplicación completa y funcional.

La arquitectura modular y el uso de tecnologías modernas permiten una fácil extensión y personalización del sistema según las necesidades específicas de cada municipio o comunidad.

==========

CONTACTO Y SOPORTE

Para preguntas, sugerencias o reportar problemas:

• Email: soporte@reparamibarrio.com • GitHub: github.com/reparamibarrio/proyecto • Twitter: @reparamibarrio • Web: www.reparamibarrio.com

Documentación online: docs.reparamibarrio.com Comunidad: forum.reparamibarrio.com

==========

LICENCIA

Este proyecto está bajo licencia MIT.

Copyright (c) 2025 Repara mi Barrio

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

==========

CRÉDITOS Y ATRIBUCIONES

• Diseño original: Figma - Repara mi Barrio Prototipo • Iconos: Lucide React • Componentes UI: Radix UI • Framework CSS: Tailwind CSS • Imágenes de muestra: Unsplash • Build tool: Vite • Framework: React

==========

VERSIÓN DEL DOCUMENTO: 1.0 FECHA: Octubre 2025 ÚLTIMA ACTUALIZACIÓN: 01/10/2025